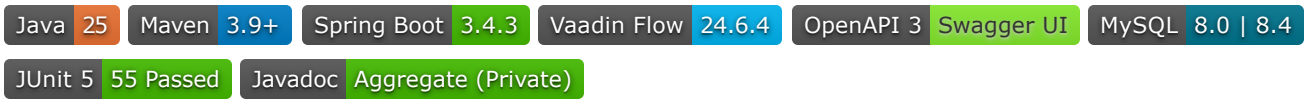
README - Albero di Regressione

Albero di Regressione (Java)



Sistema distribuito in Java per l'apprendimento induttivo, la serializzazione binaria e la predizione supervisionata mediante **Alberi di Regressione** (Regression Trees). Il sistema integra un'architettura duale: 1. **Modalità Socket TCP Concorrente (Console):** demone multithread su porta `8080` con client terminale interattivo. 2. **Modalità Web Enterprise (REST & Vaadin):** backend Spring Boot 3.4.3 con REST API stateless, documentazione interattiva OpenAPI/Swagger UI su porta `8080` (`http://localhost:8080/swagger-ui.html`) e interfaccia web reattiva in puro Java con Vaadin Flow su porta `8081` (`http://localhost:8081`).

Il core matematico include persistenza relazionale su database MySQL (`MapDB`), calcolo ottimizzato della varianza (SSE), partizionamento in-place con quicksort e supporto ad attributi discreti e continui.

📑 Indice dei Contenuti

- Panoramica del Progetto
- Architettura del Sistema
- Indice della Documentazione
- Guida Rapida all'Avvio
  - 1. Configurazione Rapida Database MySQL
  - 2. Compilazione del Progetto
  - 3. Modalità Web: Avvio Server REST e Client Vaadin
  - 4. Modalità Socket TCP (Console Tradizionale)
  - 5. Esecuzione dei Test Automatizzati
- Risoluzione Automatica delle Dipendenze con Maven
- Generazione della Documentazione (PDF e Javadoc)

- Struttura della Repository
- Autore e Licenza

Panoramica del Progetto

Il software affronta problemi di regressione numerica supervisionata: - **Induzione Ricorsiva dell'Albero:** Minimizzazione progressiva della Somma dei Quadrati degli Errori (SSE) sui nodi interni. - **Split Polimorfici:** - *Attributi Discreti (* `DiscreteNode` *):* partizionamento multi-ramo con scansione lineare post-ordinamento contiguo. - *Attributi Continui (* `ContinuousNode` *):* ricerca della soglia di taglio binario ottima τ minimizzando la somma degli SSE dei due rami candidati ($\leq \tau$ e $> \tau$). - **Navigazione Stateless:** Metodo `navigate(List<Integer> path)` che consente l'ispezione dell'albero e la predizione passo-passo via HTTP/REST senza dipendenza da connessioni socket aperte. - **Interfaccia Web Vaadin Flow:** Dashboard in puro Java su porta `8081` con navigazione visuale dei bivi decisionali e visualizzazione istantanea delle regole IF-THEN. - **Specifiche OpenAPI 3 e Swagger UI:** Endpoint documentati interattivamente su `/swagger-ui.html` . - **Ottimizzazione di Memoria:** Ordinamento in-place mediante **quicksort** con profondità stack $O(\log_2 n)$. - **Server Socket Concorrente (** `MultiServer` **):** Gestione multithread su socket TCP. - **Persistenza Binaria (** `.dmp` **):** Serializzazione del modello su disco per interrogazioni immediate a freddo.

Architettura del Sistema

```
graph TB
    subgraph Frontend Tier
        CLI[Client Console - Client.Main]
        WEB[Vaadin Flow Web UI :8081 - Client.ClientApplication]
    end

    subgraph Backend Tier
        MS[MultiServer - Socket Listener :8080]
        REST[Spring Boot REST API :8080 - TreeController]
        SWAGGER[Swagger UI / OpenAPI 3 :8080]
        RT[RegressionTree Engine - tree.*]

        MS --> RT
        REST --> RT
        REST -.-> SWAGGER
    end

    subgraph Data Tier
        DB[(MySQL MapDB)]
        FS[(Local Storage: .dmp Archives)]
        RT --> DB
        RT --> FS
    end

    CLI <== TCP Socket ==> MS
    WEB <== HTTP/JSON RestClient ==> REST
```

Indice della Documentazione

L'intera documentazione tecnica e operativa del progetto è disponibile sia in formato sorgente **Markdown (`.md`)** sia in formato ad alta definizione **PDF (`docs/pdf/*.pdf`)**, affiancata dalla documentazione API aggregata **Javadoc**:

Documento	Descrizione	Versione Markdown	Versione PDF
Guida Tecnica Completa	Architettura three-tier, formulazione matematica SSE, quicksort in-place, split discreti/continui, protocollo socket, REST API e Vaadin.	docs/guida-tecnica.md	docs/pdf/guida-tecnica.pdf
Guida Utente e Installazione	Script SQL `MapDB`, credenziali `MapUser` / `map`, uso interfaccia Web Vaadin Flow, Swagger UI ed esecuzione CLI.	docs/guida-utente-installazione.md	docs/pdf/guida-utente-installazione.pdf
Documentazione della Suite di Test	Matrice di tracciabilità requisiti, 55 test automatizzati (JUnit 5, Mockito, Spring Boot Test E2E, RestClient) a 0 errori.	docs/documentazione-test.md	docs/pdf/documentazione-test.pdf
Architettura Software, UML e OOP	Teoria OOP Appice, 6 classi sintattiche, matrice di Cook, diagrammi UML dei package	docs/architettura-uml-oop.md	docs/pdf/architettura-uml-oop.pdf

Documento	Descrizione	Versione Markdown	Versione PDF
	`dto`, Spring e Vaadin.		
Documentazione Javadoc	Documentazione API completa di tutti i package (inclusi metodi e campi privati).	docs/javadoc/index.html	—
README di Progetto	Panoramica sintetica, architettura, badge, comandi di build e avvio.	README.md	docs/pdf/README.pdf

---

Guida Rapida all'Avvio

1. Configurazione Rapida Database MySQL

Assicurarsi che il servizio MySQL sia attivo su `localhost:3306`, quindi eseguire lo script di bootstrap:

```
CREATE DATABASE IF NOT EXISTS MapDB;
CREATE USER IF NOT EXISTS 'MapUser'@'localhost' IDENTIFIED BY 'map';
GRANT ALL PRIVILEGES ON MapDB.* TO 'MapUser'@'localhost';
FLUSH PRIVILEGES;

USE MapDB;

DROP TABLE IF EXISTS provaC;
CREATE TABLE provaC (
    X VARCHAR(10) NOT NULL,
    Y DOUBLE NOT NULL,
    C DOUBLE NOT NULL
);

INSERT INTO provaC (X, Y, C) VALUES
('A', 2.0, 1.0), ('A', 2.0, 1.0), ('A', 1.0, 1.0), ('A', 2.0, 1.0),
('A', 5.0, 1.5), ('A', 5.0, 1.5), ('A', 6.0, 1.5), ('B', 6.0, 10.0),
('A', 6.0, 1.5), ('A', 6.0, 1.5), ('B', 10.0, 10.0), ('B', 5.0, 10.0),
('B', 12.0, 10.0), ('B', 14.0, 10.0), ('A', 1.0, 1.0);
```

2. Compilazione del Progetto

Dalla radice della repository:

```
mvn clean compile
```

3. Modalità Web: Avvio Server REST e Client Vaadin

3.1 Avvio Backend Spring Boot REST API (Porta 8080):

```
mvn -pl Server spring-boot:run
```

- **Swagger UI:** `http://localhost:8080/swagger-ui.html`
- **OpenAPI 3 Spec (JSON):** `http://localhost:8080/v3/api-docs`

3.2 Avvio Frontend Web Vaadin Flow (Porta 8081):

In un secondo terminale:

```
mvn -pl Client spring-boot:run
```

- **Interfaccia Web:** `http://localhost:8081`

4. Modalità Socket TCP (Console Tradizionale)

Avvio del Server Socket:

```
java -cp "Server/target/classes;C:/Users/marco/.m2/repository/com/mysql/mysql-connector-j/8.4.0/mysql-connector-
        j-8.4.0.jar" Main
```

Avvio del Client Console:

In un altro terminale:

```
java -cp "Client/target/classes" Client.Main
```

5. Esecuzione dei Test Automatizzati

Esecuzione dell'intera suite (55 test unitari, d'integrazione, socket ed end-to-end):

```
mvn clean test
```

Generazione della Documentazione (PDF e Javadoc)

È possibile rigenerare l'intera documentazione tecnica e i file PDF ad alta risoluzione eseguendo lo script PowerShell dedicato:

```
powershell -ExecutionPolicy Bypass -File scripts/build-docs.ps1
```

Lo script esegue: 1. `mvn javadoc:aggregate` per compilare il Javadoc aggregato con visibilità `private` in `docs/javadoc/`. 2. Rendering vettoriale dei PDF ad alta risoluzione in `docs/pdf/` per `guida-tecnica.pdf`, `guida-utente-installazione.pdf` e `README.pdf`.

Struttura della Repository

```
.
├── Client/                     # Modulo Client a riga di comando
│   ├── pom.xml                 # Configurazione Maven del client
│   └── src/main/java/
│       ├── Client/             # Main.java (punto di ingresso CLI)
│       └── utility/            # Keyboard.java (gestione robusta input da console)
├── Server/                     # Modulo Server e Motore di Regressione
│   ├── pom.xml                 # Dipendenze (mysql-connector-j 8.4.0)
│   └── src/main/java/
│       ├── Main.java           # Entrypoint Server (avvio MultiServer su porta 8080)
│       ├── data/               # Strutture dati: Data, Attribute, DiscreteAttribute, ContinuousAttribute
│       ├── database/           # Accesso JDBC: DbAccess, TableSchema, TableData, Example, Column
│       ├── server/             # Networking: MultiServer, ServerOneClient, eccezioni
│       └── tree/               # Motore ad albero: RegressionTree, Node, SplitNode, LeafNode, ...
├── docs/                       # Documentazione completa di progetto
│   ├── guida-tecnica.md        # Guida Tecnica Completa
│   ├── guida-utente-installazione.md # Guida Utente ed Esempi
│   ├── javadoc/                # Javadoc HTML aggregato (apidocs)
│   └── pdf/                    # Versioni PDF ad alta risoluzione
│       ├── guida-tecnica.pdf
│       ├── guida-utente-installazione.pdf
│       └── README.pdf
├── scripts/
│   └── build-docs.ps1          # Script PowerShell per rigenerare PDF e Javadoc
├── pom.xml                     # Root POM aggregator (Java 25, javadoc plugin 3.11.2)
└── README.md                   # Panoramica e indice principale
```

Autore e Licenza

- **Autore:** Marco Diana
- **Progetto:** Albero di Regressione (Corso di Metodi Avanzati di Programmazione)
- **Licenza:** Progetto ad uso didattico e accademico.